

实验 3-嵌入式软件串口自动化测试

1. 实验介绍

本实验基于 Code_05_Auto 嵌入式工程，要求学生在阅读理解现有 STM32 程序功能的基础上，对核心 C 代码进行适度插装，加入语句覆盖率统计与执行时间测量功能，并将修改后的程序烧录到 STM32F103 目标板；同时在 PC 端使用 Python 及串口通信实现一个简化的“串口调试器 + 自动化测试驱动”程序，能够自动生成并发送测试数据、接收与保存板端返回结果，最后结合 pytest 与 pytest-html 等测试工具，对功能正确性、覆盖率与时间性能进行自动化测试与报告生成，从而完整体验“读代码—插装—烧录—串口联调—自动化测试—结果分析”的嵌入式软件测试流程。

2. 实验目的

（1）掌握在已有嵌入式工程 Code_05_Auto 基础上阅读、理解和修改 C 程序的能力，能根据需求对程序进行插装（语句覆盖率计数、执行时间测量）而不破坏原有功能。

（2）理解语句覆盖率、分支覆盖率等基本概念，能够在 STM32 目标板上实现简单的覆盖率统计结构，并通过串口查询或回传统计结果。

（3）掌握在 STM32F103 上获取程序执行时间的方法，例如利用 SysTick、DWT 计数器或 HAL_GetTick 等机制，设计统一的 Get_us() / Get_ms() 等时间接口，完成单次调用或多次调用的执行时间测量。

（4）掌握在 PC 端使用 Python + 串口库（pyserial）实现“简化串口调试器”的方法，包括端口配置、数据发送与接收、协议封装以及结果解析。

（5）掌握如何在 PC 端随机生成测试数据，自动发送到目标板驱动被测函数运行，实时采集并保存目标板返回的测试结果到数据文件（如 CSV / TXT），为后续统计分析与绘图提供数据基础。

（6）掌握使用 pytest 和 pytest-html 等测试套件，在 PC 端构建结构化测

试脚本，对串口交互接口 / 目标板业务功能进行自动化回归测试，并自动生成可视化 HTML 测试报告。

（7）培养学生的实验报告撰写能力与系统调试能力，强调“从需求到测试报告”的完整工程思维。

3. 实验环境

（1）**硬件：**嵌入式软件测试教学实验系统板-A1、USB 口串口线、USB 口 ST-Link 烧录线、电源；Win 系统开发电脑（至少 2 个 USB 接口）

（2）**软件**

- 代码编辑器：Keil MDK v535、Cursor/VScode Copilot（推荐辅助使用）
- 烧写工具：ST-Link
- 串口软件：XCOM V3.0.exe
- 其它工具：Python、Matlab 自动化脚本环境
- 自动化环境：Python 3.x、pyserial、pytest、pytest-html、matplotlib、numpy
- 其它工具：文本编辑器、表格工具（记录与分析测试结果）

4. 实验任务

4.1. 实验任务 1 — 阅读与分析基础工程

目标：在 Code_05_Auto 工程基础上，系统阅读核心源文件 main.c 及相关业务逻辑代码，弄清楚当前 STM32 程序完成了哪些功能、输入输出是什么、关键数据结构与函数之间如何协同工作，尤其是与串口通信和业务处理相关的部分。通过理解 TEST 等串口命令的格式、响应字符串内容以及整体执行流程，为后续插装、自动化测试和结果分析打下坚实基础。

验收标准：学生能够清楚当前工程中“被测业务函数”的功能、输入参数含义、输出结果含义，以及主要分支或处理路径，为后续测试用例设计提供依据。

4.2. 实验任务 2 — 程序插装与语句覆盖率统计

目标：在不改变原有功能正确性的前提下，对核心 C 代码进行“插装”改造：在关键分支或语句块前后加入覆盖率计数语句，并在合适位置加入执行时间测量代码，构建一个能在 STM32F103 上运行的语句覆盖率与性能观测机制。通过设计简单的串口命令（如 COVERAGE、RESET 等），让 PC 端能够远程查看各语句块的命中次数和部分性能数据。

验收标准：经插装修改并重新编译、烧录后的程序在目标板上运行稳定，能够正常响应 TEST/COVERAGE/RESET 等基础串口命令，记录并总结程序插装与语句覆盖率、函数运行的时间性能。

4.3. 实验任务 3 — PC 端串口自动化测试驱动程序设计

目标：在 PC 端使用 Python 编写一个“串口自动化测试驱动”程序，基于已封装好的 SerialDevice 类或类似接口，实现自动打开/关闭串口、批量产生测试数据、按照协议格式组装并发送测试指令到 STM32 板，并持续接收、解析并保存板端返回的功能与性能数据。通过自动化脚本代替人工点击串口调试工具，显著提高测试效率与数据采集的完整性。

验收标准：在指定串口号和波特率下自动完成连接，并在一次运行中连续向目标板发送多组（例如 ≥ 100 组）测试输入；脚本能够正确解析每次返回的数据，记录并分析，为后续的覆盖率分析和性能曲线绘制提供可靠数据源。

4.4. 实验任务 4 — 自动化测试用例设计与 pytest 测试套件

目标：在前一任务的 Python 串口驱动基础上，引入 pytest 和 pytest-html 等测试框架，将原本“脚本式”的测试整理为结构化的测试用例和测试类，使对 STM32 板业务功能的测试变成可维护、可回归、可视化的自动化测试套件。通过合理划分测试场景（如正常输入、边界值、非法输入、覆盖率查询等），构建适用于教学和扩展的嵌入式串口自动化测试框架样例。

验收标准：能顺利执行多条测试用例，生成至少一份 HTML 测试报告；学生能说明如何在修改板端代码后，重复运行同一组 pytest 测试来进行回归验证

（pytest 测试框架基本结果如下图）。

```
test_sensor_system.py::TestBasicFunctionality::test_valid_input PASSED
test_sensor_system.py::TestBasicFunctionality::test_temp_boundaries[-51-error] PASSED
test_sensor_system.py::TestBasicFunctionality::test_temp_boundaries[151-error] PASSED
===== 3 passed in 1.59s =====
```

4.5. 实验任务 5 — 源码记录、复现与报告提交

目标：综合前面所有实验环节，从功能正确性、语句/分支覆盖率、执行时间特性以及自动化测试流程等多个维度，对嵌入式软件串口自动化测试的实施效果进行分析与总结，形成一份结构完整、论证清晰的实验报告。通过数据图表和文字说明，帮助学生反思测试用例设计的充分性、自动化工具的价值以及在真实工程中如何扩展本实验思路。

验收标准：实验报告中包含对典型测试结果的整理与分析，包括：代表性测试用例及其预期/实际结果、覆盖率结果示意（可以是表格或示意图）、执行时间随输入规模或参数变化的曲线或散点图等；能够对比“手工串口测试”和“Python + pytest 自动化测试”在效率、可重复性、可维护性方面的差异；报告结构清晰，至少包含“实验目的、环境与对象、设计与实现、测试与结果、覆盖率与性能分析、问题与改进、总结与心得”等核心部分，能从整体上概括嵌入式软件串口自动化测试的流程和经验教训。